Laboratory Work 1: Concurrency in Practice

Goal. Measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload. Decide from your own measurements when the work is CPU-bound and when it is I/O-bound, observe a race condition and the cost of fixing it, and find where parallel speed-up stops.

Keywords: concurrency · CPU-bound · I/O-bound · multithreading · producer–consumer · race condition · speed-up

1 Requirements

- Any programming language that provides OS threads.
- One program; every task and configuration is selected by command-line parameters.
- A Git repository with the source code (for example, on GitHub or GitLab); its link is given in the report.
- A USB flash drive, for Task 2.
- Record your hardware: CPU model, number of physical cores and logical processors, RAM, disk type, the USB drive and its USB version, operating system, language and runtime version.

2 Data

Generate a text file `numbers.txt` containing 50 000 000 random integers from −1 000 000 to 1 000 000, one number per line. Any random generator may be used. This file is the input for every task.

For Task 2, split `numbers.txt`, in order, into 8 files of 6 250 000 numbers each.

Correctness check. In every task, the numbers of negative, positive and zero values must add up to 50 000 000 and must be the same in all tasks.

3 Task 1: One file

Goal. How CPU-bound work scales with the number of threads, what synchronisation costs, and how a producer shares work with consumers through a queue.

Task 1a: Counting with threads

Count the negative, positive and zero numbers in `numbers.txt` with 1, 2, 4, 8, 16, 32 and 64 threads created by your program. Divide the file into as many parts as there are threads; each thread reads and parses only its own part. No line may be counted twice or skipped, and reading the whole file on a single thread is not allowed. Measure the execution time and the peak memory.

Task 1b: Three ways to count

Load all the numbers into an integer array (not measured). With as many threads as your processor has logical processors, count the signs in three ways:

1. shared counters, without synchronisation;
2. shared counters protected by a lock or mutex;
3. a separate counter for each thread, added together after all threads finish.

Record the time and the totals of each version. Run version 1 three times.

Task 1c: Producer and consumers

One thread, the *reader*, reads `numbers.txt` line by line and puts the lines, in batches of 10 000, into a thread-safe queue that holds at most 100 batches: a blocking queue or channel from your language's library, or your own queue protected by a lock. The other threads, the *workers*, repeatedly take the next batch and count its numbers. When the queue is full, the reader waits; when it is empty, the workers wait. At the end of the file the reader closes the queue, and the workers stop once it is empty. Run it with 1, 2, 4, 8, 16, 32 and 64 workers and measure the execution time and the peak memory.

Questions for the report.

1. At which number of threads does the speed-up stop growing? Relate it to your physical cores and logical processors.
2. Why do 32 and 64 threads not help? Are they slower, and why?
3. Does the peak memory depend on the number of threads? Why?
4. Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.
5. Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?
6. What would happen with batches of a single line? And with a queue that has no size limit?

4 Task 2: Many files on slow storage

Goal. Decide from measurements whether the work is CPU-bound or I/O-bound when the files are on slow storage.

Copy the 8 files to a USB flash drive; in this task the files are read only from the drive. **Before every run, eject the drive, unplug it and plug it back in**, so that no data comes from the file cache. On macOS, also disable Spotlight indexing for the drive.

A file is the unit of work: one thread reads it in a single operation into one byte buffer (not as a list of lines), parses it and counts its numbers. The files are divided among the threads as evenly as possible. Measure the execution time and the peak memory with 1, 2 and 4 threads; one run per configuration is enough.

Questions for the report.

1. Compare the time on one thread with Task 1a on one thread, which parses the same numbers from your own disk. On the USB drive, is the work CPU-bound or I/O-bound? Support the answer with numbers.
2. On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

5 Report

The report contains:

1. the link to the Git repository with the source code;
2. the hardware description;
3. for every task, charts of the execution time, the speed-up relative to one thread, and the peak memory against the number of threads, followed by the answers to the questions of that task;
4. a conclusion: on your machine, which work was CPU-bound, which was I/O-bound, and how you know.

The repository contains the source code and instructions to build and run every task.

6 Control questions

1. What is the difference between a physical core and a logical processor? Where did it show in your charts?
2. How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?
3. Explain step by step how unsynchronised shared counters lose updates.
4. Why are per-thread counters faster than shared counters protected by a lock?
5. What does “I/O-bound” mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?
6. What is the difference between dividing the work into equal parts in advance and taking it from a queue? When is each one better?

7 Bonus

Implement the tasks in more than one programming language, and compare the languages in the report: how threads, locks and queues are written in each of them, whether their threads really run in parallel, and the time and the memory of the same task. Keep all the versions in the same repository. This earns bonus points at the defense.

8 Grading

Each grade requires everything above it in Table 1. A task counts only if its totals are correct, its questions are answered in the report, and you can explain its code at the defense (code understanding).

Table 1. Requirements for each grade.

Grade	Completed
5	Task 1a
6	Task 1b
7	Task 1c
8	Task 2
9	At least half of the control questions, answered at the defense
10	All control questions, answered at the defense